Optimization Models for Targeted Offers in Direct Marketing

Implementation and computational study of the exact formulations

after F. Talla Nobibon, R. Leus, F.C.R. Spieksma, *EJOR* 210 (2011) 670–683

Judecir Cavalcante
ist431224

PhD Programme in Engineering and Management
Department of Engineering and Management
Instituto Superior Técnico, Universidade de Lisboa

Optimization and Applications
Prof. José Rui Figueira
Homework 3

Problem and models follow [1]. **Blue** *marks the presenter's implementation, experiments and interpretation.*

Outline

1. Problem and data
2. Formulations
3. Implementation
4. Experimental design
5. Experiment 1: small instances
6. Experiment 2: group S1
7. Summary

The problem

- A **direct-marketing campaign** offers products to selected customers. The paper addresses *retention* campaigns, which seek additional revenue from existing customers of a bank [1].
- Response models estimate the expected return $p_{ij} = r_{ij}\, DFV_{ij}$ of offering product j to customer i (acceptance probability times value to the firm); these estimates are **inputs** of the model.

Product targeting problem

Decide which products to offer to which customers so as to maximize the expected net profit of the campaign, subject to four business requirements:

1. a corporate **hurdle rate** R on the return on investment, ;
2. a **budget** B_j per product, because each business unit funds its own product;
3. a maximum number M_i of **offers per customer**;
4. a **minimum-quantity commitment** (MQC) O_j: a product used in the campaign is offered to at least O_j customers.

A **fixed cost** f_j is incurred whenever product j is used.

The problem is strongly NP-hard; even finding a solution with positive profit is NP-hard [1].

Implementation: Instance generation

Generation rules of Section 5.1 of [1], compared with the implementation. Notation: $C_j = \sum_i c_{ij}$, $\bar{M} = \sum_i M_i/n$, $F_j = \frac{O_j}{m(1+R)} \sum_i \left[p_{ij} - (1+R)c_{ij}\right]$.

Parameter	Meaning	Rule in the paper	Implementation	Check
c_{ij}	cost of an offer	integer, uniform in $\{1, 2, 3\}$	same	✓
p_{ij}	return of an offer	integer, uniform in $\{0, \ldots, 16\}$	same	✓
R	hurdle rate	5%, 10% or 15%	same	✓
m, n	customers, products	S1: $m = 100$, $n \in \{5, 10, 15\}$	same	✓
M_i (s)	offers per customer	uniform in $\{1, \ldots, n/5\}$	uniform in $\{1, \ldots, \max(1, \lfloor n/5 \rfloor)\}$	✓
M_i (ℓ)	offers per customer	uniform in $\{\lceil n/3 \rceil, \ldots, \lceil 2n/3 \rceil\}$	same	✓
O_j	MQC	uniform in $\{\lceil \bar{M} \rceil, \ldots, \lceil 2\bar{M} \rceil\}$	uniform in $\{\lceil \bar{M} \rceil, \ldots, \lfloor 2\bar{M} \rfloor\}$	**deviation**
B_j (min, max)	budget	$\lfloor O_j C_j/m \rfloor$ and $\lceil 2\bar{M} C_j/m \rceil$	same	✓
B_j (random)	budget	integer between $O_j C_j/m$ and $2C_j/n$	integer between the min and max budgets	**deviation**
f_j	fixed cost	integer between $F_j/2$ and F_j (rounding not stated)	$\lceil F_j/2 \rceil$ to $\lfloor F_j \rfloor$; fallback if empty	✓

Validation of the generator

- Both deviations are documented: the cap on O_j prevents a minimum budget above the maximum, and $2C_j/n$ is treated as a typographical error. All 104 instance files are reproduced exactly; the f_j fallback is never used.
- **Undocumented:** the random stream restarts with the same seed in every call. For a given n, instances with different R and M_i variant share p_{ij}, c_{ij}; with the same M_i variant they also share M_i, O_j. The paper does not say whether its instances are independent.

Model M1

Variables: $x_{ij} = 1$ if product j is offered to customer i; $y_j = 1$ if product j is used in the campaign.

$$\max \sum_{i,j} (p_{ij} - c_{ij})\, x_{ij} - \sum_j f_j\, y_j \qquad (1)\ \text{net profit}$$

$$\text{s.t.} \sum_{i,j} p_{ij} x_{ij} \;\geq\; (1+R)\Big[\sum_{i,j} c_{ij} x_{ij} + \sum_j f_j y_j\Big] \qquad (2)\ \text{hurdle rate}$$

$$\sum_i c_{ij} x_{ij} \leq B_j, \qquad j = 1, \ldots, n \quad (3)\ \text{budget}$$

$$\sum_j x_{ij} \leq M_i, \qquad i = 1, \ldots, m \quad (4)\ \text{offers per customer}$$

$$\sum_i x_{ij} \leq m\, y_j, \qquad j = 1, \ldots, n \quad (5)\ \text{linking}$$

$$\sum_i x_{ij} \geq O_j\, y_j, \qquad j = 1, \ldots, n \quad (6)\ \text{MQC}$$

$$x_{ij},\ y_j \in \{0, 1\}. \qquad (7)$$

- (M1) has $mn + n$ binary variables and $1 + m + 3n$ constraints. Constraints (5) and (6) together state that a product is either unused or offered to at least O_j customers.
- In the LP relaxation, (5) $\sum_i x_{ij} \leq m\, y_j$ allows $y_j = \sum_i x_{ij}/m$, so the fixed cost f_j is charged only at a rate of only f_j/m per offer. This weakens the LP bound of (M1).

Model M2

(M2) replaces the aggregated linking constraints (5) $\sum_i x_{ij} \leq m\, y_j$ by their **disaggregated** form

$$x_{ij} \leq y_j, \qquad i = 1, \ldots, m,\ j = 1, \ldots, n. \qquad (8)$$

Proposition 3 [1]

The convex hull of the feasible solutions of (M1) equals the convex hull of the solutions satisfying (2)–(4), (6) and (8), with $0 \leq y_j \leq 1$ and $x_{ij} \in \{0, 1\}$.

- Summing (8) over i gives (5); hence every LP solution of (M2) is an LP solution of (M1), and $z^{LP}_{(M2)} \leq z^{LP}_{(M1)}$.
- (M2) has the same variables as (M1) and $mn - n$ additional constraints (mn constraints (8) replace the n constraints (5)).
- In the paper, (M2) is the formulation solved with CPLEX 10.2.

Remark *(implementation)*

The implementation keeps $y_j \in \{0, 1\}$ in the integer version of (M2). By Proposition 3 this does not change the optimal value, and the LP relaxation is the same, since x and y are relaxed to $[0, 1]$ in both cases.

Model M3

For each product j, let $S_{pj}, p = 1, \ldots, k_j$, be the sets of customers with $|S_{pj}| \geq O_j$ and $\sum_{i \in S_{pj}} c_{ij} \leq B_j$. The binary variable z_{pj} equals 1 if product j is offered exactly to S_{pj}.

$$\max \sum_j \sum_p \left[\sum_{i \in S_{pj}} (p_{ij} - c_{ij}) - f_j \right] z_{pj} \tag{9}$$

$$\text{s.t.} \sum_j \sum_p \left[\sum_{i \in S_{pj}} \big(p_{ij} - (1+R)c_{ij}\big) - (1+R) f_j \right] z_{pj} \geq 0 \tag{10}$$ hurdle rate

$$\sum_j \sum_{p:\, i \in S_{pj}} z_{pj} \leq M_i, \qquad i = 1, \ldots, m \tag{11}$$ offers per customer

$$\sum_{p=1}^{k_j} z_{pj} \leq 1, \qquad j = 1, \ldots, n \tag{12}$$ one set per product

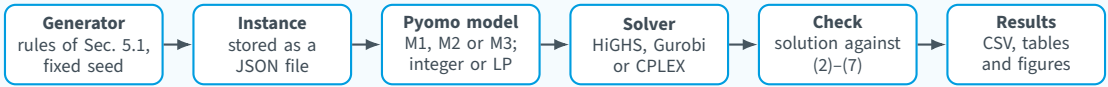
$$z_{pj} \in \{0, 1\}. \tag{13}$$

- Budget (3) and MQC (6) are built into the definition of the columns. (M3) results from a Dantzig–Wolfe decomposition, and $z^\star \leq z^{LP}_{(M3)} \leq z^{LP}_{(M2)}$ [1].
- The number of columns grows as $O(n\, 2^m)$. The paper solves (M3) by column generation within branch-and-price.

Remark *(implementation)*

The implementation enumerates all admissible columns explicitly and is therefore restricted to $m \leq 12$. Branch-and-price is not implemented. The solution of (M3) is mapped back to (x, y) and checked against (2)–(7).

Implementation: Overview



- **Pyomo as a common modelling layer.** Each formulation is written once, in algebraic form close to the paper. The same model is sent to HiGHS, Gurobi or CPLEX by changing only the solver name; Pyomo translates the model and returns the solution, the status and the bound in a uniform way. Only the time limit is passed to the solvers.
- **One model builder for M1 and M2.** The two formulations differ only in the linking constraints, (5) or (8). The LP relaxation is obtained from the same builder by changing the domain of x and y from $\{0, 1\}$ to $[0, 1]$.
- **M3 in two steps.** The admissible columns are enumerated first (budget and MQC checked for each subset of customers), and the set-covering model is then built from them. Its solution is mapped back to (x, y).
- **Safeguards.** Every integer solution, from any model and solver, is checked against (2)–(7) by a function that does not use Pyomo. Unit tests cover the 3-customer example, a brute-force enumeration of all solutions and the reproducibility of the generator.
- **Reproducibility.** Instances are generated from fixed seeds and saved before solving; each run writes one CSV row per instance, model and solver.

Design: Experiments

Experiment 1 *(small instances, M1, M2 and M3)*

- **50 instances**, $m = 4, \ldots, 12$ and $n = 2, \ldots, 5$; R, M_i variant and budget variant cycled deterministically; seeds 4000–4052.
- 53 candidates screened with (M1) and HiGHS; 3 with optimum 0 discarded, because the relative gap is undefined.
- Integer model and LP relaxation of M1, M2, M3; time limit 300 s.
- $50 \times 3 \times 3 = 450$ integer runs and 450 LP runs.

Experiment 2 *(group S1 of the paper, M1 and M2)*

- **54 instances**, $m = 100$: $n \in \{5, 10, 15\} \times 3$ values of $R \times 2$ M_i variants $\times$ 3 budgets; seed 1.
- (M3) is excluded: explicit enumeration is limited to $m \leq 12$.
- Integer model and LP relaxation of M1 and M2; time limit 3600 s.
- $54 \times 2 \times 3 = 324$ integer runs and 324 LP runs.

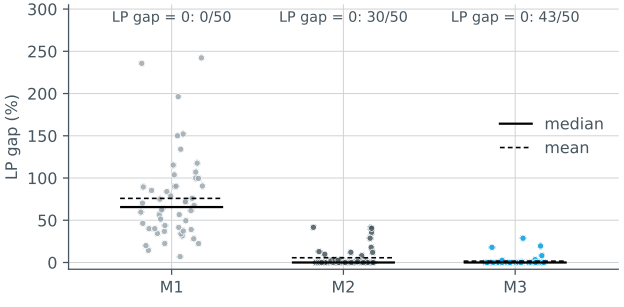
Computational environment *(identical for both experiments)*

Machine	Intel Core i7-8565U @ 1.80 GHz (4 cores, 8 threads), 15.8 GB RAM, Windows 10 Home (10.0.19045)
Modelling	Python 3.12.10, Pyomo 6.10.1
Solvers	HiGHS 1.15.1 (`highspy`, in-memory `appsi` interface); Gurobi 13.0.3 (`gurobipy`, LP-file interface); CPLEX 22.2.0.0 (Interactive Optimizer, LP-file interface). Only the time limit is set; all other parameters keep their default values.
Time	wall-clock time of the Pyomo `solve` call: includes the interface overhead, excludes model construction and the enumeration of the M3 columns.

Correctness: every integer solution is checked against (2)–(7) by an independent function. The 10 unit tests pass, including a 3-customer example with 8 admissible columns and a unique optimum 11, confirmed by enumerating all $2^6 = 64$ offer matrices.

Experiment 1: LP bounds

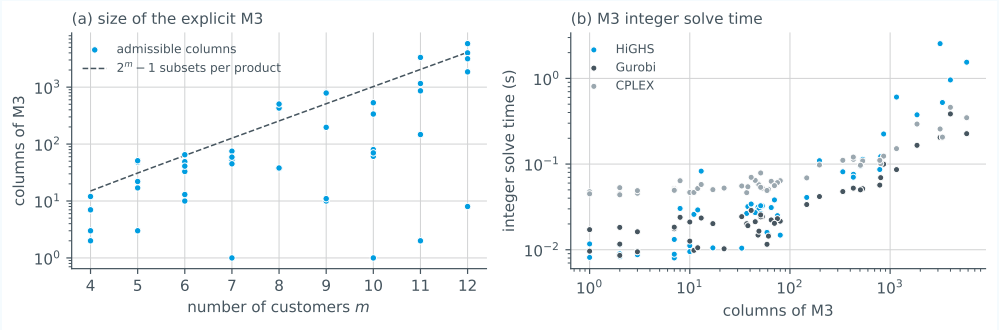
- All 450 integer runs and 450 LP runs end with proven optimality. For every instance, the integer optimum is the same for the three formulations and the three solvers (largest difference 6×10^{-13}).
- The LP gap $100\,(z^{LP} - z^{\star})/z^{\star}$ is identical across solvers (largest difference 9×10^{-14} percentage points); it depends on the formulation only. The values below use the Gurobi runs.
- The ordering $z^{\star} \leq z^{LP}_{(M3)} \leq z^{LP}_{(M2)} \leq z^{LP}_{(M1)}$ holds in all 50 instances; the inequality between M2 and M1 is strict in 50/50, and between M3 and M2 in 16/50.



	LP gap (%)			LP gap
Model	mean	median	max	$= 0$
M1	75.90	65.63	242.22	0/50
M2	5.63	0.00	41.67	30/50
M3	1.63	0.00	28.75	43/50

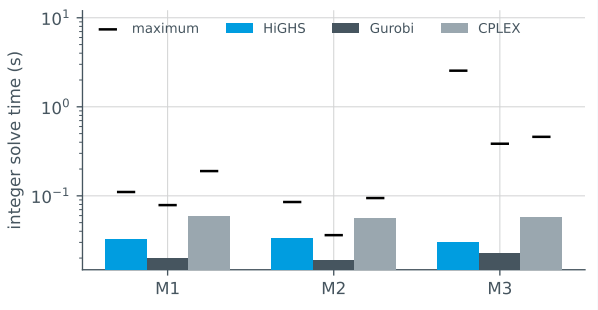
In 43 of 50 instances the LP relaxation of (M3) already gives the integer optimum; (M2) does so in 30.

Experiment 1: Size of M3



- (a) The number of admissible columns ranges from 1 to 5 809 (median 50). It is bounded by $n(2^m - 1)$, but the budget and the MQC remove a large and variable share of the subsets, so instances with the same m differ by several orders of magnitude.
- (b) The integer solve time of (M3) increases with the number of columns: Spearman correlation 0.88 (HiGHS), 0.81 (Gurobi), 0.88 (CPLEX). All times stay below 2.6 s.

Experiment 1: Solve times



Bars: median over 50 instances; marks: maximum.

		integer time (s)		
Solver	Model	median	mean	max
HiGHS	M1	0.033	0.036	0.110
	M2	0.033	0.033	0.085
	M3	0.030	0.166	2.546
Gurobi	M1	0.020	0.022	0.079
	M2	0.019	0.020	0.036
	M3	0.023	0.049	0.385
CPLEX	M1	0.060	0.064	0.190
	M2	0.056	0.058	0.094
	M3	0.058	0.092	0.460

- The medians are similar across formulations. On the 6 instances with at least 1 000 columns, the median time of (M3) is 0.78 s (HiGHS), 0.21 s (Gurobi) and 0.28 s (CPLEX), against at most 0.07 s for (M1) and (M2).

Remark *(interpretation)*

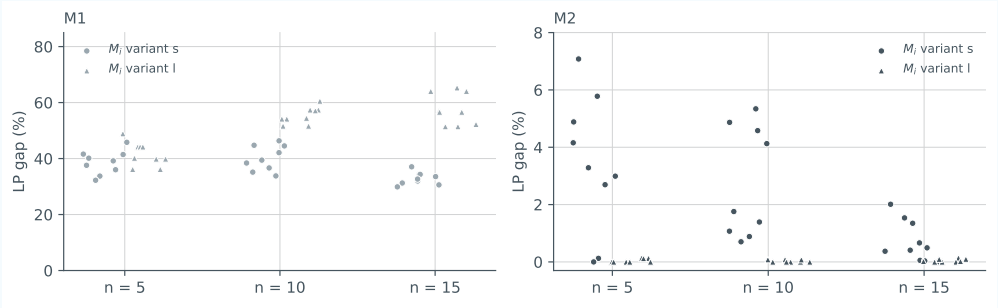
At this scale, the differences between solvers are of the order of hundredths of a second. They are likely to reflect the interface (in-memory for HiGHS, LP file for Gurobi and CPLEX) more than the algorithms, so they do not rank the solvers.

Experiment 1: Conclusions

1. **The three formulations are correctly implemented.** They reach the same proven optimum on all 50 instances with the three solvers, and every solution satisfies (2)–(7).
2. **The theoretical ordering of the LP bounds is confirmed empirically**: $z^\star \leq z^{LP}_{(M3)} \leq z^{LP}_{(M2)} \leq z^{LP}_{(M1)}$ in all instances.
3. **Disaggregation is the main gain.** The mean LP gap falls from 75.90% (M1) to 5.63% (M2); the set-covering bound reduces it further to 1.63%, and closes it completely in 43 of 50 instances.
4. **The strength of (M3) has a size cost.** The explicit model can have thousands of columns already for $m \leq 12$, and its solve time grows with the number of columns. For realistic m it requires column generation, as in the paper.
5. **The LP bound depends on the formulation, not on the solver**; the solve times of these small instances do not discriminate between solvers.

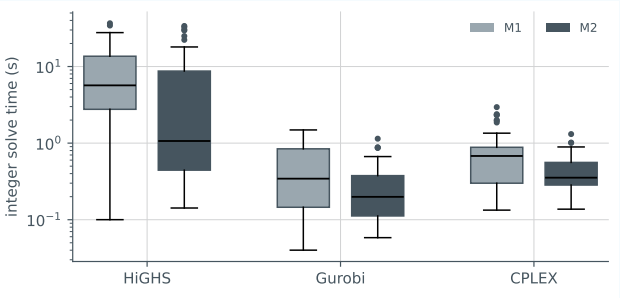
Experiment 2: LP bounds

- All 324 integer runs and 324 LP runs end with proven optimality; the final MIP gap is 0. The optimum of each instance is the same for both models and the three solvers (largest difference 10^{-11}); it ranges from 488 to 3 446.
- LP gaps are again identical across solvers. M1: mean 44.36%, range 29.89–65.30%. M2: mean 1.18%, range 0.00–7.08%, with a zero gap in 17 of 54 instances.



For (M2), the LP gap is almost zero in the ℓ variant (mean 0.03–0.04% for every n) and larger in the s variant (mean 3.44%, 2.75%, 0.77% for $n = 5, 10, 15$).

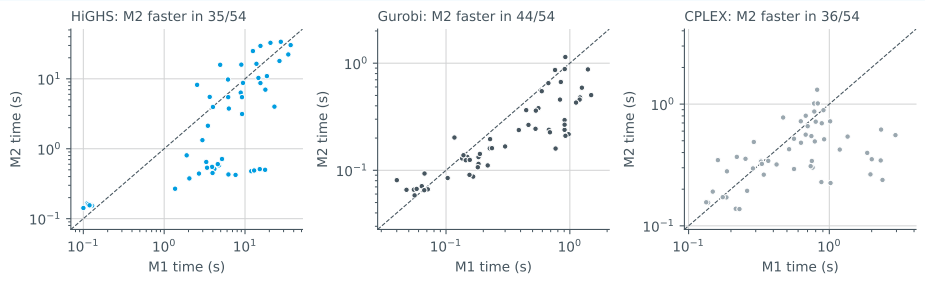
Experiment 2: Solve times



		integer time (s), 54 instances			
Solver	Model	median	mean	max	total
HiGHS	M1	5.68	8.85	36.71	477.8
	M2	1.07	6.58	33.85	355.5
Gurobi	M1	0.34	0.49	1.49	26.6
	M2	0.20	0.28	1.14	15.3
CPLEX	M1	0.68	0.78	2.95	42.2
	M2	0.35	0.46	1.31	24.5

- Every instance is solved in less than 37 s; with Gurobi and CPLEX, in less than 3 s.
- Gurobi is the fastest solver in 47 of 54 instances for (M1) and in 46 of 54 for (M2); CPLEX in the others. Geometric mean time ratio with respect to Gurobi: CPLEX 1.85 (M1) and 1.92 (M2); HiGHS 12.39 (M1) and 8.67 (M2).
- Instances with the s variant of M_i take longer: with HiGHS on (M2), mean 4.27–20.25 s against 0.16–1.88 s for ℓ.

Experiment 2: M1 versus M2



Each point is one instance; points below the diagonal are instances solved faster with (M2).

	HiGHS	Gurobi	CPLEX
Instances where M2 is faster	35/54	44/54	36/54
Geometric mean of time(M2)/time(M1)	0.46	0.66	0.69

Remark *(interpretation)*

(M2) is faster on average with every solver, but not on every instance. The modern solvers presumably strengthen (M1) by their own presolve and cuts, which would explain why its LP gap of about 44% translates into much smaller differences in solve time. The logs needed to confirm this were not recorded.

Experiment 2: Comparison with the paper

n	M_i	LP gap of (M2), mean (%) paper [1]	LP gap of (M2), mean (%) this study	integer time of (M2), mean (s) paper, CPLEX 10.2	integer time of (M2), mean (s) HiGHS	integer time of (M2), mean (s) Gurobi	integer time of (M2), mean (s) CPLEX 22.2
5	s	8.33	3.44	57.39	4.27	0.14	0.43
5	ℓ	6.51	0.04	3.91	0.16	0.07	0.23
10	s	3.11	2.75	27.90	20.25	0.43	0.80
10	ℓ	3.26	0.03	6.05	0.56	0.13	0.35
15	s	2.96	0.77	22.57	12.38	0.62	0.55
15	ℓ	1.28	0.04	3.22	1.88	0.31	0.37

Paper values: Table 4 (LP gap of the basic formulation) and Table 5 (CPLEX on the basic formulation, which the text identifies as (M2)); each cell is the mean of 9 instances, as here. Paper hardware: Pentium 2.8 GHz, 1.49 GB RAM.

- **Agreement:** in both studies, instances with the s variant of M_i are the harder ones, and all S1 instances are solved to optimality with (M2).
- **Difference:** the LP gap of (M2) is smaller here, especially for the ℓ variant (0.03–0.04% against 1.28–6.51%).

Remark *(caution)*

The instances are not the same: the paper's files are no longer available, the generator deviates in O_j and in the random budget, and the instances share data across R and M_i variants. The times also differ in hardware and solver version by more than 15 years. The comparison therefore shows only qualitative agreement.

Experiment 2: Conclusions

1. **Group S1 is easy for current solvers.** All 54 instances are solved to proven optimality with both formulations and all three solvers; the largest time is 36.7 s (HiGHS) and below 3 s with Gurobi and CPLEX.
2. **(M2) dominates (M1) in bound quality**: mean LP gap 1.18% against 44.36%, consistent with Experiment 1 and with Proposition 3.
3. **(M2) is also faster on average**, with a geometric mean time ratio of 0.46 to 0.69 depending on the solver, but it is not faster on every instance.
4. **The choice of solver matters more than the choice between M1 and M2 at** $m = 100$: HiGHS is about 9 to 12 times slower than Gurobi in geometric mean, while CPLEX is about 2 times slower.
5. **The s variant of** M_i **is the harder case**, as in the paper; the qualitative conclusions of the paper for S1 are reproduced, though not its exact numbers.

Summary: Conclusions and limitations

Conclusions

- The implementation reproduces the three exact formulations of the paper, validated by unit tests, an independent feasibility check and the agreement of 3 formulations $\times$ 3 solvers on every optimum.
- The empirical ordering of the LP bounds matches the theory: (M3) is the strongest and (M1) by far the weakest. The disaggregated linking constraints (8) account for most of the improvement.
- For the instance sizes tested, the compact model (M2) with a commercial MIP solver suffices for proven optimality: at most 1.3 s per instance with Gurobi or CPLEX.

Limitations

- (M3) is solved only by explicit enumeration ($m \leq 12$); column generation and branch-and-price, which make (M3) usable for $m = 100$, are not implemented.
- Only group S1 was run; S2, S3 and the larger groups, where the paper reports failures of the exact methods, remain untested.
- The generator has two documented deviations and shares data across instances; one seed only per configuration.
- Times are end-to-end Pyomo times under different interfaces, with default solver settings and no solver logs or node counts.

Thank you

Questions?

Judecir Cavalcante (ist431224)
PhD Programme in Engineering and Management
Department of Engineering and Management
Instituto Superior Técnico, Universidade de Lisboa

Optimization and Applications
Prof. José Rui Figueira
Homework 3

Reference

[1] F. Talla Nobibon, R. Leus, F.C.R. Spieksma. Optimization models for targeted offers in direct marketing: Exact and heuristic algorithms. *European Journal of Operational Research*, 210(3):670–683, 2011.